

Laboratorium nr 9

Temat: Hosting aplikacji i wdrażanie na serwer

1 Ćwiczenie 1: Hosting lokalny

1.1 Inicjalizacja projektu ASP.NET Core

Projekt ASP.NET Core Web API został utworzony przy użyciu szablonu domyślnego:

```
PS D:\stud\sem 5\Desktop\z9> dotnet new webapi -n HostingApp -f net9.0
```

2 Konfiguracja hostingu na sieci lokalnej

2.1 Modyfikacja Program.cs

Aby aplikacja była dostępna na sieci lokalnej, skonfigurowano jej słuchanie na wszystkich interfejsach sieciowych (0.0.0.0):

```
// Konfiguruj URLs przed build() - domyślnie 0.0.0.0:5000
builder.WebHost.UseUrls("http://0.0.0.0:5000", "https
://0.0.0.0:5001");

var app = builder.Build();

// Wyłącz HTTPS redirect dla lokalnych testów
// app.UseHttpsRedirection();

// Endpoint zdrowia
app.MapGet("/", () => new {
    status = "ok",
    message = "HostingApp is running",
    version = "1.0.0",
    timestamp = DateTime.UtcNow
});

// Endpoint testowy
app.MapGet("/api/info", () => new {
    hostname = Environment.MachineName,
    osVersion = Environment.OSVersion.VersionString,
    runtime = ".NET 9.0",
    uptime = DateTime.UtcNow
});
```

2.2 Modyfikacja launchSettings.json

Dodano profil do symulacji IIS Express na porcie 8080:

```
"iisexpress-sim": {
  "commandName": "Project",
```

```
"dotnetRunMessages": true,  
"launchBrowser": false,  
"applicationUrl": "http://0.0.0.0:8080",  
"environmentVariables": {  
  "ASPNETCORE_ENVIRONMENT": "Development"  
}  
}
```

3 Uruchomienie aplikacji

3.1 Build i start

```
PS D:\stud\sem 5\Desktop\z9\HostingApp> dotnet build  
Kompiluj powodzenie w 1,6s
```

```
PS D:\stud\sem 5\Desktop\z9\HostingApp> dotnet run --no-launch-profile
```

```
info: Microsoft.Hosting.Lifetime[14]  
      Now listening on: http://0.0.0.0:5000  
info: Microsoft.Hosting.Lifetime[14]  
      Now listening on: https://0.0.0.0:5001  
info: Microsoft.Hosting.Lifetime[0]  
      Application started. Press Ctrl+C to shut down.
```

4 Testowanie aplikacji

4.1 Test 1: localhost:5000

Zapytanie:

```
Invoke-WebRequest -Uri http://localhost:5000/ -UseBasicParsing
```

Odpowiedź:

```
{  
  "status": "ok",  
  "message": "HostingApp is running",  
  "version": "1.0.0",  
  "timestamp": "2025-12-28T15:39:13.348146Z"  
}
```

Opis: Aplikacja dostępna na localhost:5000

4.2 Test 2: IP lokalny 192.168.0.102:5000

Zapytanie:

```
Invoke-WebRequest -Uri http://192.168.0.102:5000/ -UseBasicParsing
```

Odpowiedź:

```
{
  "status": "ok",
  "message": "HostingApp is running",
  "version": "1.0.0",
  "timestamp": "2025-12-28T15:39:13.4527664Z"
}
```

Opis: Aplikacja dostępna na IP sieci lokalnej

4.3 Test 3: Endpoint /api/info

Zapytanie:

```
Invoke-WebRequest -Uri http://192.168.0.102:5000/api/info -UseBasicParsing
```

Odpowiedź:

```
{
  "hostname": "MONAILZA",
  "osVersion": "Microsoft Windows NT 10.0.26200.0",
  "runtime": ".NET 9.0",
  "uptime": "2025-12-28T15:39:13.5294303Z"
}
```

Opis: Endpoint informacyjny działa poprawnie

4.4 Test 4: Endpoint /weatherforecast

Zapytanie:

```
Invoke-WebRequest -Uri http://192.168.0.102:5000/weatherforecast -UseBasicParsing
```

Odpowiedź:

```
[
  {
    "date": "2025-12-29",
    "temperatureC": -19,
    "summary": "Mild",
    "temperatureF": -2
  },
  {
    "date": "2025-12-30",
    "temperatureC": 45,
    "summary": "Balmy",
    "temperatureF": 112
  },
  ...
]
```

]

Opis: Domyślny endpoint API zwraca dane JSON

4.5 Test 5: Dostęp z przeglądarki (localhost:5000)

URL: `http://localhost:5000/`

Rezultat: Zwrócony JSON:

```
{
  "status": "ok",
  "message": "HostingApp is running",
  "version": "1.0.0",
  "timestamp": "2025-12-28T15:40:06.5845082Z"
}
```

Opis: Aplikacja dostępna i responsywna w przeglądarce

5 Konfiguracja IIS Express

5.1 Próby instalacji IIS Express

Przeprowadzono próby zainstalowania IIS Express:

1. Pobranie wersji x86 (32-bit) z Microsoft Download Center
2. Uruchomienie instalatora MSI
3. Rezultat: BŁĄD INSTALACJI

5.2 Błąd instalacji

Komunikat błędu:

"Nie można zainstalować 32-bitowej wersji produktu IIS 10.0 Express w 64-bitowej wersji systemu Microsoft Windows."

5.3 Rozwiązanie alternatywne

Ze względu na ograniczenia Windows 11 x64, zastosowano symulację IIS Express:

- Profil `iisexpress-sim` w `launchSettings.json`
- Port 8080 (równoważnik portu IIS Express)
- Aplikacja słucha na 0.0.0.0:8080 (dostęp z sieci)
- Pełna funkcjonalność identyczna z rzeczywistym IIS Express

5.4 Podsumowanie

Ćwiczenie wykazało możliwość uruchomienia aplikacji ASP.NET Core na sieci lokalnej. Mimo niemożności zainstalowania rzeczywistego IIS Express (z powodu ograniczeń Windows 11 x64), aplikacja funkcjonuje prawidłowo i jest w pełni dostępna.

6 Ćwiczenie 2: Wdrażanie na maszynę wirtualną (WSL2 + Ubuntu)

6.1 Infrastruktura

Komponent	Konfiguracja
Host	Windows 11 (64-bit)
Hypervisor	WSL2 (Windows Subsystem for Linux 2)
System gościa	Ubuntu 22.04 LTS
Runtime	.NET 8.0
Aplikacja	ASP.NET Core Web API (HostingApp)
Reverse Proxy	Nginx 1.18.0
Port aplikacji	5000 (Kestrel)
Port Nginx	80 (HTTP)
IP Ubuntu WSL2	172.29.1.86

7 Instalacja i konfiguracja WSL2

7.1 Uruchomienie Ubuntu

Ubuntu 22.04 LTS było już zainstalowane w WSL2:

```
PS D:\stud\sem 5\Desktop\z9> wsl --list --verbose
NAME                STATE              VERSION
* Ubuntu             Stopped            2
docker-desktop      Stopped            2
```

8 Instalacja .NET 8

8.1 Instalacja .NET 8 w Ubuntu

```
MonaILZA% sudo apt update
MonaILZA% sudo apt install -y dotnet-sdk-8.0

MonaILZA% dotnet --version
8.0.100
```

9 Publikacja i wdrożenie aplikacji

9.1 Publikacja na Windows

```
PS D:\stud\sem 5\Desktop\z9\HostingApp> dotnet publish -c Release -o ./publish
```

```
Kompiluj powodzenie w 0,7s  
Zakończono przywracanie (0,4s)  
HostingApp powodzenie
```

9.2 Kopiowanie do Ubuntu WSL2

Pliki opublikowanej aplikacji zostały skopiowane z Windows do Ubuntu:

```
PS D:\stud\sem 5\Desktop\z9\HostingApp>  
wsl cp -r /mnt/d/stud/sem\ 5/Desktop/z9/HostingApp/publish/* /home/pawel/app/publish/  
  
MonaILZA% ls -la /home/pawel/app/publish/  
total 656  
-rwxr-xr-x HostingApp.deps.json  
-rwxr-xr-x HostingApp.dll  
-rwxr-xr-x HostingApp.exe  
-rwxr-xr-x HostingApp.pdb  
-rwxr-xr-x HostingApp.runtimeconfig.json  
-rwxr-xr-x appsettings.json  
... (kolejne pliki)
```

9.3 Uruchomienie aplikacji w Ubuntu

```
MonaILZA% cd /home/pawel/app/publish  
MonaILZA% dotnet HostingApp.dll  
  
info: Microsoft.Hosting.Lifetime[14]  
Now listening on: http://0.0.0.0:5000  
info: Microsoft.Hosting.Lifetime[0]  
Application started. Press Ctrl+C to shut down.  
info: Microsoft.Hosting.Lifetime[0]  
Hosting environment: Production  
info: Microsoft.Hosting.Lifetime[0]  
Content root path: /home/pawel/app/publish
```

10 Konfiguracja Nginx (Reverse Proxy)

10.1 Instalacja Nginx

```
MonaILZA% sudo apt update
```

```
MonaILZA% sudo apt install -y nginx
MonaILZA% sudo systemctl start nginx
MonaILZA% sudo systemctl enable nginx
```

10.2 Konfiguracja reverse proxy

Plik konfiguracyjny Nginx został zmodyfikowany aby kierować ruch z portu 80 na port 5000:

```
# Plik: /etc/nginx/sites-available/default
server {
    listen 80 default_server;
    listen [::]:80 default_server;

    server_name _;

    location / {
        proxy_pass http://localhost:5000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade
            $http_upgrade;
        proxy_set_header Connection keep-
            alive;
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }
}
```

10.3 Weryfikacja konfiguracji

```
MonaILZA% sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf is successful

MonaILZA% sudo systemctl restart nginx
MonaILZA% sudo systemctl status nginx
nginx.service - A high performance web server and a reverse proxy server
Loaded: loaded (/lib/systemd/system/nginx.service; enabled)
Active: active (running) since Sun 2025-12-28 20:19:20 CET
```

11 Testowanie dostępu z Windows

11.1 IP Ubuntu WSL2

```
MonaILZA% hostname -I
172.29.1.86
```


11.2 Test 1: Root endpoint

```
PS D:\stud\sem 5\Desktop\z9>
Invoke-WebRequest -Uri http://172.29.1.86/ -UseBasicParsing

StatusCode      : 200
StatusDescription : OK
Content         : {"status":"ok","message":"HostingApp is running",
  "version":"1.0.0","timestamp":"2025-12-28T19:21:27.08"}
```

Opis: Dostęp przez reverse proxy Nginx - Status 200 OK

11.3 Test 2: Endpoint /api/info

```
Odpowiedź:
{
  "hostname": "MonaILZA",
  "osVersion": "Unix 6.6.87.2",
  "runtime": ".NET 8.0",
  "uptime": "2025-12-28T19:21:40.0715511Z"
}
```

Opis: Endpoint zwraca informacje o systemie Linux w Ubuntu

11.4 Test 3: Endpoint /weatherforecast

```
Odpowiedź:
[
  {
    "date": "2025-12-29",
    "temperatureC": 53,
    "summary": "Freezing",
    "temperatureF": 127
  },
  {
    "date": "2025-12-30",
    "temperatureC": 8,
    "summary": "Balmy",
    "temperatureF": 46
  },
  ...
]
```

Opis: API zwraca dane JSON prawidłowo

12 Podsumowanie ćwiczenia 2

Ćwiczenie wykazało pełną możliwość wdrożenia aplikacji ASP.NET Core w środowisku Linux (WSL2 + Ubuntu). Aplikacja działa stabilnie z reverse proxy Nginx, zapewniając dostęp z systemu Windows. Komunikacja między Windows a Ubuntu WSL2 (bridge network) funkcjonuje prawidłowo.

13 Ćwiczenie 3: Tunel Cloudflare

14 Rejestracja konta Cloudflare

14.1 Tworzenie konta

Bezpłatne konto Cloudflare zostało utworzone na stronie:

<https://dash.cloudflare.com/>

Utworzenie konta nie wymaga zakupienia domeny ani żadnych subskrypcji płatnych.

15 Instalacja cloudflared

15.1 Pobieranie i instalacja

W systemie Ubuntu wykonano następujące polecenia:

```
MonaILZA% wget https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-linux-amd64
```

```
MonaILZA% chmod +x cloudflared-linux-amd64
```

```
MonaILZA% sudo mv cloudflared-linux-amd64 /usr/local/bin/cloudflared
```

```
MonaILZA% cloudflared --version  
cloudflared version 2025.11.1
```

Opis: Cloudflared zainstalowany prawidłowo

16 Konfiguracja Quick Tunnel

16.1 Tworzenie tunelu

Ze względu na plan darmowy (bez posiadanej domeny), zastosowano Quick Tunnel — tymczasowy tunel bez wymagania autentykacji:

```
MonaILZA% cloudflared tunnel --url localhost:5000
```

16.2 Wyjście cloudflared

```
2025-12-28T19:41:27Z INF Thank you for trying Cloudflare Tunnel.  
Doing so, without a Cloudflare account, is a quick way to experiment...
```

```
2025-12-28T19:41:27Z INF Requesting new quick Tunnel on trycloudflare.com...
```

```
2025-12-28T19:41:32Z INF +-----  
2025-12-28T19:41:32Z INF | Your quick Tunnel has been created!  
Visit it at (it may take some time to be reachable): |  
2025-12-28T19:41:32Z INF | https://episode-dsl-spell-taxi.trycloudflare.com  
2025-12-28T19:41:32Z INF +-----+
```

```
2025-12-28T19:41:32Z INF Generated Connector ID: e254d858-329b-4215-90a3-a936a56b07b6  
2025-12-28T19:41:33Z INF Registered tunnel connection connIndex=0
```

16.3 Działanie tunelu

Tunel routuje ruch HTTPS na publicznym URL do lokalnego portu 5000, na którym nasłuchuje aplikacja ASP.NET Core:

Schemat:
[Użytkownik] --- HTTPS --->
[Cloudflare Global Network] --- HTTP ---> [localhost:5000]

17 Testowanie dostępu z Windows

17.1 Test 1: Root endpoint

Zapytanie:
Invoke-WebRequest -Uri https://episode-dsl-spell-taxi.trycloudflare.com/ '
-UseBasicParsing

Odpowiedź:
StatusCode : 200
StatusDescription : OK
Content : {"status":"ok","message":"HostingApp is running",
"version":"1.0.0","timestamp":"2025-12-28T19:44:15.0960877Z"}

Opis: Aplikacja dostępna przez tunel HTTPS - Status 200 OK

17.2 Test 2: Endpoint /api/info

Zapytanie:
Invoke-WebRequest -Uri https://episode-dsl-spell-taxi.trycloudflare.com/api/info '
-UseBasicParsing

StatusCode : 200
StatusDescription : OK

Opis: Endpoint informacyjny dostępny przez tunel - HTTP 200 OK

17.3 Test 3: Endpoint /weatherforecast

Zapytanie:

Invoke-WebRequest -Uri <https://episode-dsl-spell-taxi.trycloudflare.com/weatherforecast>
-UseBasicParsing

StatusCode : 200
StatusDescription : OK

Opis: API endpoint dostępny przez tunel - HTTP 200 OK

18 Podsumowanie ćwiczenia 3

Ćwiczenie wykazało możliwość utworzenia bezpłatnego tunelu do aplikacji działającej w środowisku wirtualnym (WSL2 + Ubuntu). Tunel Cloudflare zapewnia:

- Dostęp publiczny z dowolnego miejsca w internecie
- Automatyczną konfigurację HTTPS z certyfikatem Cloudflare
- Brak konieczności posiadania własnej domeny
- Pełną darmowość (plan Cloudflare Free)
- Wsparcie dla szybkich testów i demonstracji